Software Requirements Specification (SRS)

Project: AI-Powered Personalized Recommendation System for Sustainable Kitchen Management
Version: 1.0
Prepared by: Bü■ra Topdemir, Fatma Zehra Bestil, Ay■e Sümeyra Mert, Betül Korkmaz
Institution: Konya Food and Agriculture University
Date: January 14, 2026

1. Introduction

1.1 Purpose

The primary purpose of this document is to define the technical, functional, and operational requirements of the "AI-Powered Smart Recipe Recommendation System." The project provides a technological solution that enables users to utilize ingredients in their kitchens most efficiently, with the aim of reducing household food waste. The system aims to optimize kitchen decision-making processes and promote sustainable consumption habits using artificial intelligence and image processing technologies.

1.2 Document Conventions

Requirements in this document are labeled in the "REQ-X" format. Technical terms are specified in parentheses for clarity.

1.3 Target Audience and Reading Suggestions

This document is prepared for software developers, data scientists, project advisor Prof. Dr. Meltem Huri Baturay, and the TÜB■TAK evaluation panel. Section 2 should be reviewed for the general operation of the system, and Sections 3 and 4 for technical details.

1.4 Product Scope

This system is an AI-based mobile platform that digitizes kitchen management. The key features within scope are:

- **Smart Data Entry:** Provides a structure that allows users to manually enter ingredients or automatically identify them from photos using the YOLOv object recognition model.
- **AI-Powered Recipe Generation:** Creates personalized, creative, and applicable recipes by analyzing identified ingredients using OpenAI GPT models.
- **Nutritional Analysis:** Calculates estimated calorie and macronutrient values for generated recipes.
- **Hybrid Recommendation System:** Provides increasingly accurate kitchen recommendations over time by learning user preferences and taste.
- **Social Interaction:** Enables users to rate recipes, comment, and create a social kitchen ecosystem.
- **Multi-Platform Support:** The application developed with Flutter operates simultaneously on Android and iOS devices through Python (Flask/FastAPI) backend and Firebase database.

1.5 References

- TÜB■TAK 2209-A Research Proposal Form (2025/1st Period)
- COMP1002 - Advanced Python Course Project Proposal
- IEEE Std 830-1998 Software Requirements Specification Standard

2. Overall Description

2.1 Product Perspective

This system is an independent mobile software product. Unlike traditional recipe applications, it adopts an "ingredient-to-recipe" focused approach. It provides users with a dynamic kitchen assistant experience by working integrated with various cloud-based AI services and database solutions.

2.2 Product Functions

- **Ingredient Entry:** Adding ingredients through manual text entry or camera-based object recognition.
- **Dynamic Recipe Generation:** Creating original recipes by artificial intelligence based on selected ingredients and category.

- **Personalization:** Recommendation system based on user history.
- **Nutritional Information:** Display of calorie and nutritional values per recipe.
- **Social Sharing:** Commenting on recipes, rating, and sharing visuals.
- **Point System:** Reward mechanism that increases in-app interaction.

2.3 User Classes and Characteristics

The system is designed for the following user classes:

- **Home Cooks (High Priority):** Regular cooking adults, workers, and students. Their basic needs are quick meal planning, utilizing available ingredients, and time savings. They use all basic functions of the system.
- **Health-Conscious Users (Medium Priority):** Users who focus on nutrition, calorie tracking, and healthy ingredients. Their basic needs are detailed nutritional value display and healthy recipe filters.
- **Kitchen Enthusiasts (Low Priority):** Users who love trying new recipes and seek creativity in the kitchen. Their basic needs are recipe diversity, advanced filtering features, and recipe saving/organizing functions.

Classification Differences: - All classes require authentication and basic recipe generation. - While health-conscious users prioritize nutritional data features, enthusiast users use favorite management and history features more frequently.

2.4 Operating Environment

The hardware, software, and network requirements necessary for the stable operation of the system are defined below:

- **Device Requirements:**
- Active camera support for image recognition feature.
- Stable internet access (Wi-Fi or Mobile Data).
- Android 8.0+ or iOS 12.0+ operating system support.
- **Server Environment (Backend):**
- Python Version: Python 3.8 or newer versions.
- Web Framework: Use of Flask 2.0+ or FastAPI 0.68+.
- **Cloud and External Services:**

- Data Management: Firebase Cloud Firestore (Data storage and synchronization).
- Authentication: Firebase Authentication.
- Artificial Intelligence: OpenAI GPT API (Cloud-based recipe generation).
- Object Recognition: YOLOv model positioned on backend server or as an inference service.

2.5 Design and Implementation Constraints

Factors limiting the development and operation of the system are:

- **Internet Dependency:** Since all AI functions of the system (OpenAI and Firebase) operate cloud-based, there is a requirement for an active internet connection; the application cannot function fully in offline mode.
- **Hardware and Environment Sensitivity:** The success of the YOLOv model's object recognition is directly dependent on the camera quality of the mobile device used and lighting conditions in the environment.
- **API Limitations:** Rate limiting and response times that may occur depending on OpenAI API usage should be considered during the development process.

2.6 User Documentation

Documentation components to be provided to users along with the software are:

- **User Manual (Digital Booklet):** A comprehensive guide that explains the basic features of the application, Flutter-based interface usage, and ingredient entry modules (manual entry and visual recognition) step by step.
- **Visual Recognition and Camera Usage Guide:** Special instructions containing technical tips and correct shooting angles on how to properly introduce refrigerator or available ingredients to the system through photos using the YOLOv model.
- **Online Help (In-App Help):** An interactive FAQ section accessible within the application when users get stuck (e.g., at the moment of "what should I cook?" indecision), explaining the AI bot and dynamic recipe creation processes.
- **Social Platform and Community Guide:** A guide containing usage rules and methods for social ecosystem features such as commenting on recipes, sharing photos, and interacting with other users.

2.7 Assumptions and Dependencies

- It is assumed that users photograph ingredients clearly.
- Uninterrupted accessibility of OpenAI and Firebase services is required.

3. External Interface Requirements

3.1 User Interfaces

The system's user interface will be designed in a modern UX/UI principles-compliant, minimalist, and user-friendly structure.

3.1.1 Screen Components

1. **Login/Registration Screen:** Email and password entry fields, registration form (with verification feedback), "Forgot Password" link, and clear error message display.
2. **Home Screen:** Quick access buttons for manual and camera-based ingredient entry, recent recipes, search bar, bottom navigation bar, and profile access.
3. **Ingredient Entry Screen:** Auto-complete supported text field, camera/gallery access buttons, active ingredient list (with removal option), category selection, and "Generate Recipe" button.
4. **Recipe Viewing Screen:** Recipe title and image, ingredient list with quantities, step-by-step preparation instructions, detailed nutritional values (calories, macros, etc.), cooking time/difficulty level, and "Add to Favorites" button.
5. **Recipe List Screen:** Recipe views in card structure, filtering/sorting options, infinite scroll support.
6. **Profile Screen:** User information, dietary preferences and allergy settings, favorite recipes, history records, and logout option.

3.1.2 Interface Standards

- Standard navigation patterns (Back button, Home icon).
- Consistent button styles, color palette, and typography.
- Loading indicators for asynchronous operations.
- Short notifications (toast messages) and confirmation dialogs for operation results.
- Help icon accessible from all screens.

3.1.3 Accessibility

- Screen reader support (iOS VoiceOver, Android TalkBack).
- High contrast mode and text scaling support for visually impaired users.
- Color-blind friendly color schemes.

3.1.4 Responsive Design

- Portrait (Vertical) and Landscape (Horizontal) mode support.
- Flexible (adaptive) layout that adapts to different screen sizes.
- Wide screen layouts optimized for tablet devices.

3.2 Hardware Interfaces

The interaction of the software with mobile device hardware is as follows:

- **Mobile Device Hardware Units:**
- **Camera Interface:** Access to device rear and front cameras, image capture capability. Switching between gallery and instant capture and permission management.
- **Storage Interface:** Local storage for application data and cached content. Temporary file storage area during image processing.
- **Network Interface:** Wi-Fi and Mobile Data (3G/4G/5G) connection support. Connection status detection and error management.
- **Screen and Input Interface:** Adaptation to various screen resolutions, touch screen inputs, and detection of basic gestures (swiping, tapping, zooming, etc.).
- **Sensors (Future Developments):** GPS support for location-based recommendations and accelerometer support for gesture controls are planned.
- **Hardware Requirements:**
- Camera: Required for image recognition feature.
- Storage: Minimum 100MB available space.
- RAM: Minimum 2GB RAM for smooth experience.
- Processor: Modern smartphone processor architecture.
- **Hardware Communication Protocols:**

- Camera access is provided through Flutter camera plugin.
- Network communication is conducted with standard HTTP/HTTPS protocols.
- Complex image processing processes are performed on the backend server to avoid straining the device processor.

3.3 Software Interfaces

The system uses the following interfaces to enable data transfer between various internal and external software components:

3.3.1 Backend API Interface

The server side provides RESTful API endpoints for the mobile application:

- **Authentication:** Registration (/register), Login (/login), Logout (/logout), and Profile (/profile) operations.
- **Ingredient Operations:** Image-based detection (/detect), auto-complete suggestions (/suggestions), and list validation.
- **Recipe Operations:** Recipe generation (/generate), detail querying, favorite management, recipe history, and filtering operations.
- **User Settings:** Endpoints for updating and querying preferences.

3.3.2 Data Formats and Security

- **Communication:** All requests and responses are in JSON format.
- **File Transfer:** Image uploads are performed with Multipart/form-data structure.
- **Authorization:** JWT (JSON Web Token) is used in HTTP header for security.

3.3.3 External Service Interfaces

1. **OpenAI GPT API:** JSON-based prompt-response mechanism is used for cloud-based recipe generation.
2. **Firebase Services:** Firestore NoSQL database management and user authentication processes are conducted through Firebase Admin SDK (Python).
3. **YOLOv Model:** An inference script is used that takes image input and produces JSON output containing object classes and confidence scores.

3.3.4 Database Schema

The system hosts the following collection structures on Firebase:

- **Users:** User information, email, and creation dates.
- **Recipes:** Recipe details, ingredients, instructions, and nutritional values.
- **Favorites & History:** User interactions and saved content.
- **Preferences:** Dietary preferences and allergy information.

3.3.5 Error Management

The system uses standard HTTP status codes (200, 401, 404, 500, etc.). Error responses are returned in JSON format containing error message and code to provide meaningful feedback to users or developers.

3.4 Communication Interfaces

The communication requirements and standards of the system on internal and external networks are:

- **Communication Protocols and Standards:**
- HTTP/HTTPS: All data exchange between mobile application and backend server is conducted over HTTPS (TLS 1.2 or higher) protocol.
- RESTful Architecture: Standard REST (Representational State Transfer) architecture is used for accessing and managing system resources.
- **Message and Data Formats:**
- JSON: All internal data transfer (API responses, structured data) is done in JSON (JavaScript Object Notation) format.
- Multipart/form-data: Multi-part form format supporting binary data transmission is used for photo uploads and visual ingredient recognition operations.
- **Communication Security and Encryption:**
- End-to-End Encryption: All traffic between server and client is encrypted with SSL/TLS certificates.
- JWT Authentication: Each API request must contain a JSON Web Token (JWT) verified by the server side.
- **Data Transfer Speed and Performance:**

- The system is designed to show optimum performance on 4G/5G and broadband Wi-Fi connections. API response times (excluding AI operations) are targeted to be under 500ms.
- **Synchronization Mechanisms:**
- Firebase Real-time Listeners: User preferences, favorites, and notifications are synchronized instantly across all devices through Firebase's real-time listener mechanism.
- **Email Communication:**
- SMTP-based email transmission is provided through Firebase Authentication infrastructure for registration verification, password reset, and critical notifications.

4. System Features

4.1 Visual Ingredient Recognition and Inventory Management

4.1.1 Description and Priority

This feature enables users to photograph ingredients in the kitchen (vegetables, fruits, packaged foods, etc.) using their mobile device camera and allows the system to automatically recognize these objects. The main purpose is to minimize manual data entry, accelerate user experience, and instantly record available ingredients to digital inventory to prevent food waste.

Priority: High.

4.1.2 Stimulus/Response Sequences

- **Stimulus:** User clicks the "Camera" icon on the home screen.
- **Response:** System launches camera interface and activates object recognition viewfinder.
- **Stimulus:** User takes a photo of objects.
- **Response:** System sends photo to backend server and analyzes it with YOLOv model.
- **Stimulus:** Analysis is completed.
- **Response:** System presents identified ingredients to the user as a list with confidence scores.
- **Stimulus:** User confirms or edits the identified list (adds/removes items).

- **Response:** System adds verified ingredients to temporary ingredient list.

4.1.3 Functional Requirements

- **REQ-1:** The system must be able to access the mobile device's hardware camera and capture high-resolution images.
- **REQ-2:** The YOLOv model must identify common kitchen ingredients in photos with at least 80% accuracy (confidence score).
- **REQ-3:** The system must have the ability to recognize multiple ingredients in the same frame (multi-object detection).
- **REQ-4:** The identification process (including upload and analysis) must be completed in less than 3 seconds on a standard internet connection.
- **REQ-5:** The system must inform the user and offer manual entry option if image quality is low or objects cannot be recognized.

4.2 AI-Powered Personalized Recipe Generation

4.2.1 Description and Priority

The feature of generating original recipes using OpenAI GPT architecture based on selected ingredients and category filters.

Priority: High.

4.2.2 Stimulus/Response Sequences

- **Stimulus:** User clicks the "Generate Recipe" button.
- **Response:** System sends ingredients and user preferences to GPT API as a prompt.
- **Stimulus:** API responds.
- **Response:** System displays the generated recipe in structured format (title, ingredients, steps) on screen.

4.2.3 Functional Requirements

- **REQ-6:** The system must be able to make "logical substitutions" using entered ingredients (e.g., suggesting dry onion instead of fresh onion).

- **REQ-7:** Generated recipes must be 100% compliant with user's dietary preferences (vegetarian, gluten-free, etc.).

4.3 Nutritional Value and Calorie Analysis

4.3.1 Description and Priority

Calculation and presentation of calorie and macronutrient values for each recipe to the user.

Priority: Medium.

4.3.2 Stimulus/Response Sequences

- **Stimulus:** Recipe is successfully generated.
- **Response:** System performs nutritional analysis based on portion and ingredient quantities in the content.

4.3.3 Functional Requirements

- **REQ-8:** Recipe calorie and nutritional values must be validated according to standard databases and presented to the user.

5. Other Non-Functional Requirements

5.1 Performance Requirements

Performance criteria required for stable and efficient operation of the system are defined below:

- **Response Time Requirements:**
- Recipe Generation: The recipe generation process with artificial intelligence must be completed within 10 seconds under normal load conditions. Priority: Medium
- Visual Recognition: Processing and ingredient detection for images up to 10MB must be completed within 5 seconds. Priority: Medium
- Database Queries: Standard operations such as fetching user profile or listing recipes must be responded to within 2 seconds. Priority: Medium
- Application Loading Time: Mobile application screens must load within 2 seconds under average network conditions (4G). Priority: Medium

- API Response Time: Backend API endpoints must respond within 3 seconds at the latest for standard operations. Priority: Medium
- **Capacity and Processing Requirements:**
- Concurrent Users: The system must be able to support at least 100 concurrent users without performance loss. Priority: Medium
- API Request Capacity: Backend must be able to handle at least 50 recipe generation requests per minute. Priority: Medium
- Image Processing Capacity: Backend must be able to process at least 20 image recognition requests per minute. Priority: Medium
- **Resource Usage and Optimization:**
- Application Size: The total installation size of the mobile application must not exceed 150MB. Priority: Medium
- Memory (RAM) Usage: The application must consume a maximum of 300MB RAM during normal operation. Priority: Medium
- Battery Impact: Background processes should be optimized so that application usage does not excessively consume device battery. Priority: Low
- Network Bandwidth: Image uploads and downloads should be optimized for data savings. Priority: Medium
- **Scalability Requirements:**
- Database Scalability: The system must support growth up to 10,000 users without changing the basic architecture. Priority: Low
- Horizontal Scaling: Backend architecture must be suitable for horizontal scaling in case of increasing load. Priority: Low
- **Caching Requirements:**
- Recipe Caching: Frequently accessed popular recipes should be cached to reduce database load. Priority: Low
- Image Caching: Previously analyzed images and their results should be cached to prevent reprocessing. Priority: Low

5.2 Safety Requirements

The following requirements are defined to ensure the safety of the system in terms of data, food, users, and general system operation:

- **Data Safety:**
- Data Backup: User data and recipes must be regularly backed up daily. Priority: Medium
- Data Recovery: In case of possible data loss, the system must support data restoration. Priority: Medium
- **Food Safety:**
- Allergy Warnings: The system must display prominent visual warnings for recipes containing common allergens. Priority: High
- Nutritional Information Accuracy: Nutritional values should be presented as accurately as possible; however, they must include a disclaimer that these values are estimates. Priority: Medium
- Ingredient Safety: The system must perform basic validation against potentially harmful ingredient combinations and avoid risky recipe suggestions. Priority: Low
- **User Safety:**
- Privacy Protection: Users' personal information and preferences must be protected in accordance with current data protection regulations (KVKK, etc.). Priority: High
- Content Moderation: Generated recipes must undergo basic filtering process against inappropriate or harmful content. Priority: Low
- **System Safety:**
- Error Management: The system must manage errors in a way that does not cause application crashes (gracefully). Priority: High
- Input Validation: All user inputs must be validated to prevent injection attacks and system errors. Priority: High

5.3 Security Requirements

The following requirements are defined to ensure the cybersecurity of the system and prevent unauthorized access:

- **Authentication and Authorization:**
- User Authentication: All users must log in to the system before accessing personalized features. Priority: High
- Session Management: User sessions must be managed securely using JWT tokens with validity periods. Priority: High

- Password Security: Passwords must be hashed with secure algorithms like bcrypt and never stored as plain text. Priority: High
- Token Security: JWT tokens must be signed to prevent tampering and verified on each request. Priority: High
- **Data Security:**
- Data Encryption: All traffic between mobile application and backend must be encrypted using HTTPS/TLS. Priority: High
- Data at Rest: Sensitive user data in the database must be stored encrypted. Priority: Medium
- API Key Protection: OpenAI and Firebase API keys must be securely stored on the backend and never leaked to the client side. Priority: High
- **Privacy and GDPR/KVKK Compliance:**
- Legal Compliance: The system must be fully compliant with European Union GDPR (General Data Protection Regulation) and Republic of Turkey KVKK (Personal Data Protection Law) regulations. Priority: High
- Data Minimization: The system must collect only the minimum user data necessary for service delivery. Priority: Medium
- User Consent: Explicit consent must be obtained from users for data collection and processing processes. Priority: High
- Data Deletion: Users must be able to request permanent deletion of their personal data from the system. Priority: Medium
- **Input Validation and Sanitization:**
- SQL Injection Prevention: All database queries must be made through parameterized queries or ORM layers to prevent SQL injection attacks. Priority: High
- XSS Prevention: User inputs must be sanitized to prevent Cross-Site Scripting (XSS) attacks. Priority: High
- File Upload Security: Image uploads must be validated in terms of file type and size against malware risk. Priority: Medium
- **Access Control:**
- Role-Based Access: The system must contain a basic access control structure between user and administrator roles. Priority: Low
- Rate Limiting: API endpoints must have rate limiting mechanisms to prevent abuse and DDoS attacks. Priority: Medium

5.4 Software Quality Attributes

Basic attributes defining the quality and sustainability of the system are specified below:

- **Usability:**
- Ease of Use: The application must be intuitive enough for users with basic smartphone knowledge to use without any training. Priority: High
- User Interface: The interface must follow Material Design (Android) and Human Interface Guidelines (iOS) standards for consistency. Priority: Medium
- Accessibility: The application must support screen readers and follow accessibility guidelines (WCAG 2.1 Level AA). Priority: Medium
- Error Messages: Error messages must be clear, user-friendly, and solution-oriented. Priority: Medium
- Help Documentation: In-app help and tooltips should be available for basic features. Priority: Low
- **Reliability:**
- Uptime: The system must maintain 99% uptime during operational hours. Priority: Medium
- Error Recovery: The system must recover from errors smoothly without data loss. Priority: High
- Data Consistency: The database must maintain data consistency and integrity. Priority: High
- Backup and Recovery: Regular backups must be made with tested recovery procedures. Priority: Medium
- **Maintainability:**
- Code Quality: Code must follow best practices and coding standards (PEP 8 for Python, Dart style guide for Flutter). Priority: Medium
- Documentation: Code must be well-documented with comments and docstrings. Priority: Medium
- Modularity: Code must be modular and organized for easy maintenance and updates. Priority: Medium
- Version Control: All code must be managed in a version control system (Git) with appropriate branching strategy. Priority: High
- **Portability:**

- Cross-Platform: The application must work on both iOS and Android platforms using the Flutter framework. Priority: High
- Device Compatibility: The application must support various screen sizes and resolutions. Priority: Medium
- OS Version Support: The application must support minimum operating system versions specified in Section 2.4. Priority: High
- **Testability:**
- Unit Testing: Critical functions must have unit tests with at least 70% code coverage. Priority: Medium
- Integration Testing: API endpoints and database operations must have integration tests. Priority: Medium
- User Acceptance Testing (UAT): The system must pass user acceptance tests before release. Priority: High
- **Interoperability:**
- API Standards: The backend API must follow RESTful standards for interoperability. Priority: Medium
- Data Formats: The system must use standard data formats (JSON) for data exchange. Priority: High
- **Efficiency:**
- Resource Usage: The application must use device resources (CPU, memory, battery) efficiently. Priority: Medium
- Network Optimization: API calls and data transfers must be optimized to minimize bandwidth usage. Priority: Medium

5.5 Business Rules

Basic rules determining the operation of the system and in-app decisions are defined below:

- **User Management Rules:**
- BR-1: Each user must have a unique email address for registration. Priority: High
- BR-2: Users must be at least 13 years old to create an account (COPPA compliance). Priority: High
- BR-3: Users can have only one active session per device. Priority: Medium
- **Recipe Generation Rules:**

- BR-4: At least 2 ingredients must be provided to the system for recipe generation. Priority: High
- BR-5: Generated recipes must primarily use provided ingredients (at least 70% of ingredients must match). Priority: Medium
- BR-6: Recipes containing allergens specified by the user must be excluded from recommendation lists. Priority: High
- BR-7: To manage API costs, recipe generation is limited to 10 requests per user per hour. Priority: Medium
- **Data Management Rules:**
- BR-8: User data will be kept in the system as long as the account is active. Priority: High
- BR-9: Deleted user accounts and related data must be permanently removed within 30 days. Priority: Medium
- BR-10: Recipe history will be limited to the last 50 recipes viewed per user. Priority: Low
- **Content Rules:**
- BR-11: Generated recipe content must be appropriate and safe for the general public. Priority: High
- BR-12: Nutritional values are presented as estimates; users are advised to consult nutritionists for precise dietary needs. Priority: Medium
- **Access Rules:**
- BR-13: Unregistered (anonymous) users can view sample recipes but cannot add favorites or access personalized features. Priority: Medium
- BR-14: Visual recognition feature requires camera permission; the system must request this permission appropriately. Priority: High

6. Other Requirements

6.1 Database Requirements

The data storage and management standards of the system are as follows:

- **Database Type:**

- DB-1: The system will use Firebase Cloud Firestore, which provides a NoSQL document-based structure. [Priority: High]
- **Database Schema:**
- DB-2: The database must include collections: users (profiles, preferences), recipes (recipes and metadata), favorites (favorite matches), history (viewing history), and preferences (dietary settings). [Priority: High]
- **Data Relationships:**
- DB-3: User documents must reference recipe documents through unique "Recipe ID"s. [Priority: High]
- DB-4: Favorites and history collections must consistently maintain relationships between User ID and Recipe ID. [Priority: High]
- **Data Indexing:**
- DB-5: Database indexes must be created on frequently queried fields (User ID, category, creation date). [Priority: Medium]
- **Data Backup:**
- DB-6: The database must be backed up daily and backups must be kept for 30 days. [Priority: Medium]

6.2 Internationalization Requirements

- **Language Support:**
- INT-1: The application must fully support Turkish and English languages. [Priority: Medium]
- INT-2: Users must be able to select their preferred language from application settings. [Priority: Medium]
- INT-3: AI-powered recipe generation must be able to provide content in both languages according to user preference. [Priority: Medium]
- **Localization:**
- INT-4: Date and time formats must be displayed according to the user's local settings (locale). [Priority: Low]
- INT-5: Number formats (decimal separators, etc.) must follow local conventions. [Priority: Low]

6.3 Legal and Compliance Requirements

- **Data Protection:**
- LEG-1: The system must be compliant with European Union GDPR (General Data Protection Regulation) standards. [Priority: High]
- LEG-2: The system must meet Republic of Turkey KVKK (Personal Data Protection Law) requirements. [Priority: High]
- LEG-3: The system must contain privacy policy and terms of use documents. [Priority: High]
- LEG-4: Users must approve these documents at the registration stage. [Priority: High]
- **Content Responsibility:**
- LEG-5: The system must contain a disclaimer that recipes are generated by artificial intelligence and nutritional data are estimates. [Priority: Medium]
- LEG-6: It must be stated that the system cannot be held responsible for food safety issues that may occur during recipe preparation. [Priority: Medium]

6.4 Reusability Requirements

- **Component Reusability:**
- REU-1: Backend API must be designed to be used by future web applications or different clients. [Priority: Low]
- REU-2: Visual recognition module must be in a modular structure that can be used in other projects. [Priority: Low]

6.5 Future Development Requirements (Out of Scope)

- **Planned Developments:**
- FUT-1: Social Features: Recipe sharing, user comments, and voting. [Priority: Low]
- FUT-2: Gamification: Achievement system and cooking challenges. [Priority: Low]
- FUT-3: Shopping List: Automatic list creation from recipes. [Priority: Low]
- FUT-4: Meal Planning: Calendar integration and weekly planning. [Priority: Low]
- FUT-5: Market Integration: Ability to directly order ingredients. [Priority: Low]
- FUT-6: Offline Mode: Access to saved recipes without internet. [Priority: Low]

- FUT-7: Voice Command: Ingredient entry by voice. [Priority: Low]
- FUT-8: Video Guides: Supporting recipes with video explanations. [Priority: Low]

Appendix A: Glossary

- **AI (Artificial Intelligence):** Technology that enables machines to perform tasks that typically require human intelligence, such as understanding natural language and recognizing images.
- **API (Application Programming Interface):** A set of protocols and tools for building software applications, allowing different software components to communicate.
- **Authentication:** The process of verifying the identity of a user or system.
- **Backend:** The server-side of an application that handles business logic, database operations, and API integrations.
- **Firebase:** Google's mobile and web application development platform providing backend services including database, authentication, and hosting.
- **Firestore:** Firebase's NoSQL document database for storing and syncing data.
- **Flutter:** Google's UI toolkit for building natively compiled applications for mobile, web, and desktop from a single codebase.
- **Frontend:** The client-side of an application that users interact with, typically the user interface.
- **GPT (Generative Pre-trained Transformer):** A type of AI model developed by OpenAI capable of generating human-like text based on prompts.
- **JWT (JSON Web Token):** A compact, URL-safe token format used for securely transmitting information between parties.
- **NLP (Natural Language Processing):** A branch of AI that helps computers understand, interpret, and generate human language.
- **NoSQL:** A database management system that stores data in formats other than relational tables (e.g., documents, key-value pairs).
- **REST (Representational State Transfer):** An architectural style for designing networked applications using HTTP protocols.
- **YOLOv (You Only Look Once):** A real-time object detection algorithm that can identify objects in images quickly and accurately.

- **Recipe Generation:** The process of creating recipe suggestions using AI based on provided ingredients and user preferences.
- **Ingredient Recognition:** The process of identifying food ingredients from images using computer vision technology.
- **Nutritional Information:** Data about the nutritional content of food, including calories, macronutrients (protein, carbs, fats), and micronutrients (vitamins, minerals).
- **Dietary Restrictions:** Limitations on food consumption due to health, ethical, or religious reasons (e.g., vegetarian, vegan, gluten-free, halal).
- **Meal Category:** Classification of recipes into groups such as Main Dishes, Breakfast, Desserts, Appetizers, and Beverages.
- **User Profile:** Collection of user information including preferences, dietary restrictions, allergies, and account settings.
- **Favorites:** User-saved recipes for quick access.
- **Recipe History:** Record of recipes previously viewed by the user.

Appendix B: Analysis Models

This section includes diagrams visualizing the system's architectural structure, data processing logic, and user interactions.

B.1 System Architecture Diagram

The system is built on a multi-layered architecture using Flutter on the client side, Python (Flask/FastAPI) on the server side, and Firebase for data management.

B.2 Data Flow Diagram (DFD - Level 1)

This diagram shows the flow of data between system components from the initial user input to the final output.

B.3 Use Case Diagram

The use case diagram outlines the system boundaries and the primary functions available to the user.

Appendix C: To Be Determined (TBD) List

Points that need to be clarified in later stages within the SRS scope are:

1. **TBD-1 (Nutritional Data Source):** Clear selection of the standard database (USDA, FatSecret, etc.) to be used for calorie and macronutrient calculations.
2. **TBD-2 (YOLOv Training Dataset):** Specific local food dataset on which the model will be trained to increase recognition success above 80% (e.g., vegetable types specific to Turkey).
3. **TBD-3 (API Cost Management):** Determination of monthly limits for OpenAI and Firebase usage fees according to project budget.
4. **TBD-4 (Hardware Compatibility Testing):** Minimum GPU/CPU thresholds required to optimize YOLOv inference times on low-segment Android devices.